Tribhuvan University

Faculty of Humanities and Social Sciences

Track Number

A PROJECT PROPOSAL

Submitted to
Department of Computer Application
Xavier International College

In partial fulfillment of the requirements for the Bachelor's in Computer Application

Submitted by

Sushank Karki BCA 4th Semester Symbol No: 54602170	Kushal Karki BCA 4th Semester Symbol No: 54602164

Under the Supervision of

Contents

List of Figures

Abstract

Manual and spreadsheet-based vehicle record lookup is slow, error-prone, and difficult to scale as the number of records grows. This proposal presents *Track Number*, a web-based vehicle registration lookup system that allows a user to retrieve vehicle details — including vehicle name, model, owner, and registered year — by submitting a registration number through a simple search interface. The proposed system follows a three-tier architecture [1] consisting of an HTML/CSS frontend [2], a Python backend built with the FastAPI framework [3], and a PostgreSQL relational database [4] for persistent storage. This document sets out the problem statement, objectives, methodology, feasibility analysis, system design (flowchart, data flow diagram, and entity–relationship diagram), work plan, and expected outcome for the project, which is proposed as a Bachelor of Computer Application (BCA) academic project.

Index Terms — *vehicle registration lookup; web application; FastAPI; PostgreSQL; system design; academic project proposal.*

1. Introduction

Track Number is a web-based vehicle registration lookup system developed to make vehicle identification faster and easier. The system allows users to search for vehicle details by entering the vehicle registration number, or number plate, in a search bar. Once the registration number is submitted, the system checks the stored vehicle records and displays details such as vehicle name, vehicle model, vehicle owner, and registered year.

For example, when a user enters the number plate “123ABC”, the system can display information such as vehicle name: Hayabusa X3, vehicle model: 2026, vehicle owner: Sushank Karki, and registered year: 2021. The project will be developed using Python [5], FastAPI [3], HTML, CSS [2], and a PostgreSQL database [4]. Since it is delivered as a website, it can be accessed from both mobile phones and personal computers through a standard web browser.

The main goal of this project is to reduce manual searching of vehicle records, provide a simple search interface, and organize vehicle data in a secure and systematic database. For academic demonstrations, the system will use sample vehicle records. In a real deployment, access to owner information should be restricted and controlled to protect privacy and prevent misuse.

2. Problem Statement

In many cases, vehicle information is difficult to search quickly because records may be stored manually, in spreadsheets, or across separate systems. If a person wants to know the basic details of a vehicle using its registration number, the process can take time and may require manual checking. This creates problems in record management, searching, and identification.

Track Number is proposed to solve this problem by providing a web-based platform where a user can enter a vehicle registration number and instantly view the related vehicle details if the record exists in the database. The system will help organize vehicle data, reduce manual work, improve searching speed, and make the process more accurate and convenient.

3. Objectives

The main objectives of the Track Number system are:

- To develop a simple and user-friendly vehicle registration lookup website.
- To allow users to search vehicle records using a vehicle registration number.
- To display vehicle details such as vehicle name, model number, owner name, and registered year.
- To store and manage vehicle information using a PostgreSQL database.
- To build a beginner-friendly backend API using Python and FastAPI.
- To make the website responsive so that it runs properly on both mobile and desktop browsers.

4. Methodology

4.1 Requirement Identification

Requirement identification is the process of determining the features, tools, and resources needed to develop a software system [7]. For the Track Number system, this includes a frontend for entering the registration number and viewing results, a backend API for processing the search request, and a PostgreSQL database for storing vehicle records.

The basic working process of the system is as follows: the user opens the website, enters a vehicle registration number, the backend validates the input, searches the database, and returns the vehicle information if it is available. If the number is not found, or the input is invalid, the system displays a clear message and returns the user to the input step.

4.1.1 Existing System

The existing approach to vehicle record checking is mostly manual or limited to internal record systems. A user may need to check paper files, spreadsheets, or ask an authorized person to find vehicle details. This process is time-consuming and may introduce errors when records are large or not updated properly. Searching by registration number manually is also inconvenient.

The proposed Track Number system replaces this manual approach with a simple web-based search system. The database stores the vehicle registration number and related details, and the website provides quick access through a search bar.

4.1.2 Device / Platform Requirements

- The system is a web application and does not require special hardware.
- It should run on any mobile phone, tablet, laptop, or desktop with a modern web browser.
- An internet connection is required once the system is hosted online.
- For local development, a standard computer or laptop can run the backend server and database.

4.1.3 Software Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Frontend: HTML and CSS [2], [12].
- Backend: Python with the FastAPI framework.
- Database: PostgreSQL.
- Code Editor: Visual Studio Code [8].
- Web Browser: Google Chrome, Microsoft Edge, Firefox, or Safari.

4.2 Feasibility Analysis

Feasibility studies are typically assessed along operational, technical, and economic dimensions [7]. The feasibility study for Track Number concluded that the project can be implemented successfully because the required technologies are freely available, beginner-friendly, and cost-effective. The project is suitable for academic development and can be completed using open-source tools.

4.2.1 Operational Feasibility

- The system is easy to operate because users only need to enter a vehicle registration number in the search bar.
- The result page displays vehicle information in a clear and simple format.
- The administrator can add, update, and delete vehicle records in the database.
- The system reduces manual searching and makes vehicle records look up faster.

4.2.2 Technical Feasibility

- The system can be developed using Python and FastAPI, which are well suited to building web APIs [3]
- PostgreSQL can safely and reliably store structured vehicle records [4]
- HTML and CSS can be used to build a responsive user interface [2], [6]
- The system can be tested locally and later hosted online if required.
- The technology stack is practical and beginner friendly.

4.2.3 Economic Feasibility

- The project is economically feasible because most required tools are free and open source.
- Python, FastAPI, PostgreSQL, HTML, CSS, and Visual Studio Code can be used without licensing cost [5], [8]
- No special hardware is required because it is a web-based system.
- The system can reduce the time and effort needed for manual record searching.

4.3 System Architecture

Track Number will be built as a three-tier web application [1], which separates the user interface, the application logic, and the data storage into independent layers so that each layer can be developed, tested, and modified without affecting the others. Figure 1 shows how a search request moves through the three tiers.

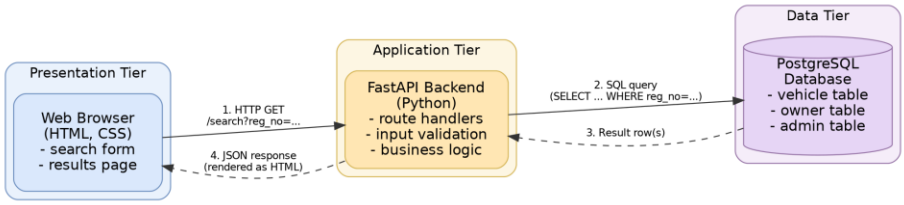

Fig. 1. Three-tier system architecture and request/response cycle for the Track Number system.

As shown in Figure 1, the browser (presentation tier) sends the registration number entered by the user as an HTTP GET request to a FastAPI route. The FastAPI backend (application tier) validates the input, builds a parameterized SQL query, and forwards it to PostgreSQL (data tier). The database returns the matching row, if any, which the backend serializes into a JSON response and the frontend renders as an HTML results page. Separating the tiers in this way keeps input validation and query logic out of the browser and keeps the database schema hidden behind the API rather than exposed directly to the client.

4.4 Development Approach

Because Track Number is a small, well-defined academic project with a fixed set of features, the authors will follow an incremental development approach [7], building and testing one working layer of the system before adding the next, rather than attempting to design and code the entire system in a single pass. This keeps a runnable version of the system available throughout development and makes it easier to isolate defects introduced at each stage. The planned build order, which maps directly onto the work plan in Section 6, is as follows:

- Design the PostgreSQL schema for the vehicle, owner, and admin tables shown in the ER diagram (Fig. 4), and populate it with sample records for demonstration.
- Implement the FastAPI backend: a route to search a vehicle by registration number, and basic admin routes to add, update, and delete records.
- Build the HTML/CSS frontend: a search form on the home page and a results page that displays the vehicle details or a "no record found" message.
- Integrate the frontend with the backend API, then test the flow end-to-end against the cases in the flowchart (Fig. 2): valid number found, valid number not found, and invalid number format.
- Review, fix defects found during testing and prepare the final project report and presentation.

At the API level, the core planned endpoints are GET /search/ {registration_number} to look up a vehicle, and POST, PUT, and DELETE routes under /admin/vehicles for the administrator to manage records. Keeping the endpoint list small and mapping each one directly to a use case from the objectives in Section 3 keeps the first working version of the system achievable within the timeframe in Section 6.

5. System Design

This section presents the visual design artifacts for the Track Number system: a process flowchart describing the search logic, a data flow diagram (DFD) describing how information moves between the user, the system, and the administrator, and an entity–relationship (ER) diagram describing the underlying database structure.

5.1 Flowchart

Figure 2 shows the process flow of the Track Number system, drawn using standard flowchart symbols and conventions [9], from the point a user opens the website to the point vehicle details are displayed or a "no record found" message is shown. Note that when the entered registration number fails the format check, control returns to the input step so the user can re-enter a valid number, rather than proceeding to the output stage.

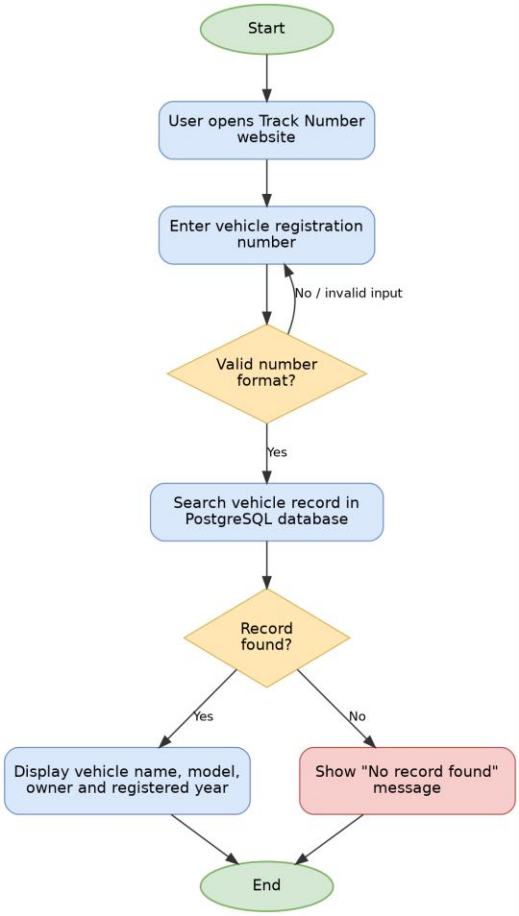

Fig. 2. Flowchart of the Track Number system.

5.2 Data Flow Diagram (DFD)

Figure 3 shows the Level-0 data flow diagram of the system, following the structured-analysis data-flow-diagram notation [10]. It illustrates how a registration number flows from the user into the Track Number system, how the system queries the vehicle database, and how an administrator manages records through a separate admin database.

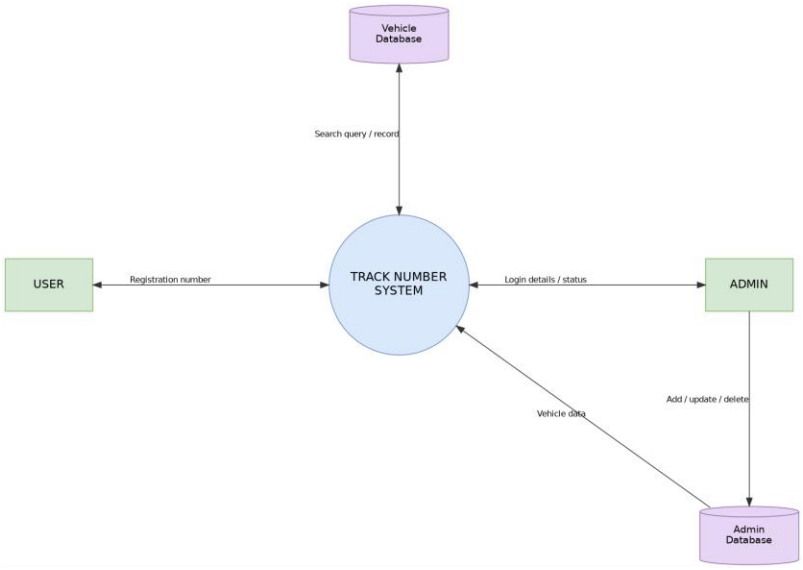

Fig. 3. Data flow diagram (DFD) for the Track Number system.

5.3 Entity–Relationship (ER) Diagram

Figure 4 shows the entity–relationship diagram for the underlying database, using the entity–relationship modeling notation [11]. The USER entity searches for a VEHICLE, each VEHICLE has an associated OWNER, and an ADMIN manages VEHICLE records.

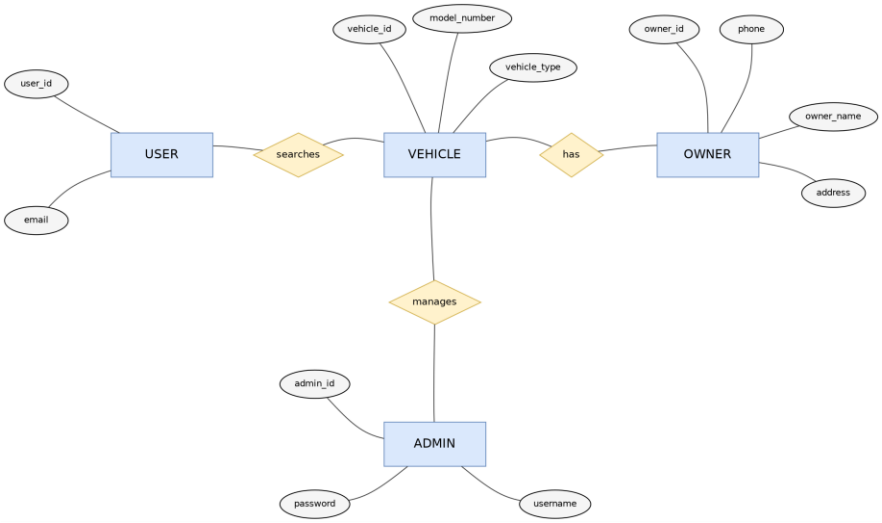

Fig. 4. Entity–relationship (ER) diagram for the Track Number system.

6. Project Timeline (Work Plan)

Figure 5 presents the proposed Gantt chart for the Track Number project as a standard horizontal-bar schedule, spanning the months of Asar, Shrawan, and Bhadra 2083 B.S. Each bar represents the duration of one project phase; phases are scheduled sequentially so that later phases can build on the outputs of earlier ones.

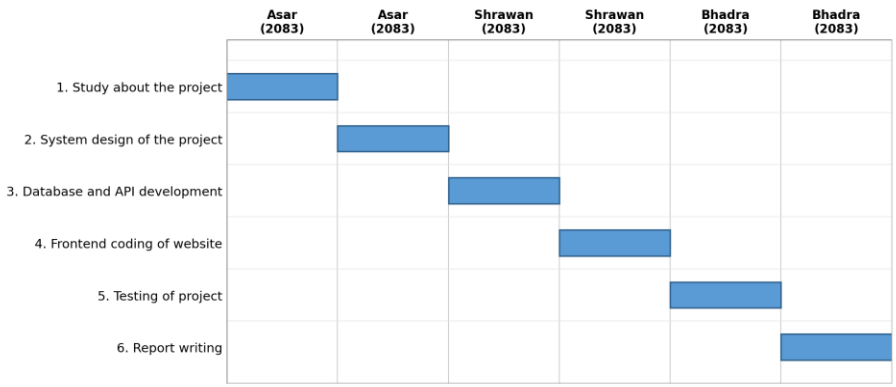

Fig. 5. Gantt chart / work plan for the Track Number system.

7. Expected Outcome

The Track Number system is expected to provide a simple and effective way to search for vehicle information using a vehicle registration number. Users will be able to open the website from a mobile phone or PC, enter the number plate, and view the vehicle details stored in the database. The system will display information such as vehicle name, vehicle model, vehicle owner, and registered year.

The system will help organize vehicle records in PostgreSQL, reduce manual searching, improve accuracy, and make vehicle identification faster. The project will also help the authors understand how frontend, backend API, and database components work together in a complete web application.

Overall, Track Number is expected to be a useful, beginner-friendly, and practical web-based vehicle record lookup system.

8. Conclusion

This proposal has presented Track Number, a web-based vehicle registration lookup system intended to replace slow, error-prone manual and spreadsheet-based record searching with a fast, structured, and beginner-friendly web application. The problem statement, objectives, methodology, feasibility analysis, and system design presented in the preceding sections show that the project is technically achievable within an academic timeframe using free and open-source tools, namely Python, FastAPI, HTML, CSS, and PostgreSQL.

The proposed design corrects the search-and-validation logic at the flowchart level, so that invalid input is returned to the user for correction rather than silently reaching the output stage, and defines a clear data flow between the user, the system, and the administrator. On completion, the project is expected to demonstrate a working three-tier web application and to give the authors practical experience integrating a frontend, a backend API, and a relational database.

As future work beyond the scope of this academic proposal, the system could be extended with user authentication and role-based access control, an audit log of administrator changes, integration with an official vehicle registration authority's records, and deployment on a production web server with HTTPS and backup procedures to make it suitable for real-world use rather than academic demonstration alone.

Acknowledgment

The authors would like to thank their project supervisor for guidance and feedback throughout the preparation of this proposal, and the Department of Computer Application, Xavier International College, for the opportunity to undertake this project as part of the Bachelor of Computer Application program at Tribhuvan University.

References

[1] Microsoft, "N-tier architecture style," Azure Architecture Center, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier. [Accessed: 4-Jul-2026].

[2] Mozilla Developer Network (MDN), "HTML and CSS — Web development documentation," Mozilla. [Online]. Available: https://developer.mozilla.org. [Accessed: 4-Jul-2026].

[3] FastAPI, "FastAPI documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com. [Accessed: 4-Jul-2026].

[4] PostgreSQL Global Development Group, "PostgreSQL documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/. [Accessed: 4-Jul-2026].

[5] Python Software Foundation, "Python documentation," 2024. [Online]. Available: https://docs.python.org/3/. [Accessed: 4-Jul-2026].

[6] E. Marcotte, "Responsive Web Design," A List Apart, May 25, 2010. [Online]. Available: https://alistapart.com/article/responsive-web-design/. [Accessed: 4-Jul-2026].

[7] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

[8] Microsoft, "Documentation for Visual Studio Code." [Online]. Available: https://code.visualstudio.com/docs. [Accessed: 4-Jul-2026].

[9] International Organization for Standardization, Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts, ISO 5807:1985, Geneva, Switzerland, 1985.

[10] T. DeMarco, Structured Analysis and System Specification. Englewood Cliffs, NJ, USA: Prentice-Hall, 1978.

[11] P. P.-S. Chen, "The entity-relationship model — toward a unified view of data," ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.

[12] W3Schools, "HTML and CSS tutorials," Refsnes Data. [Online]. Available: https://www.w3schools.com. [Accessed: 4-Jul-2026].